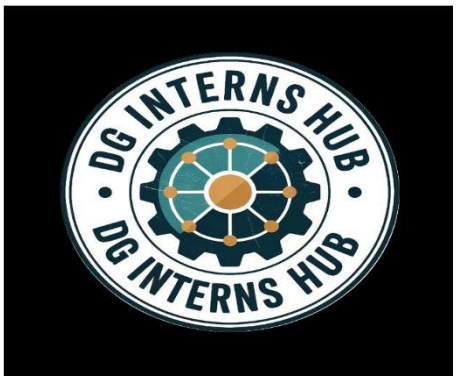


DG INTERNS HUB – CYBERSECURITY INTERNSHIP

WEEK 7 ASSIGNMENT REPORT:

VULNERABILITY ASSESSMENT AND PENETRATION TESTING (VAPT)

Web Application Testing with Burp Suite

NAME: K. JENIFA JASMINE
STUDENT ID: DG/SEPTEMBER/CYBER/240
DURATION: 3 MONTHS (1ST SEPTEMBER TO 30TH NOVEMBER)
DOMAIN: CYBERSECURITY SUBMISION
DATE: 20th OCTOBER 2026

Table of Contents

1. Introduction and Objective
2. Scope and Tools Used
3. Task 22 — Burp Suite Setup and Interception
4. Task 23 — Reconnaissance and Mapping of the Web Application
5. Task 24 — Vulnerability Testing (Manual + Burp Tools)
6. Task 25 — Directory and Endpoint Discovery
7. Task 26 — Vulnerability Report (VAPT Style)
   7.1 SQL Injection in Login Form
   7.2 Reflected Cross-Site Scripting (XSS)
   7.3 Broken Authentication / Weak Credential Policy
8. Risk Summary and Severity Matrix
9. Conclusion

1. Introduction and Objective

Vulnerability Assessment and Penetration Testing (VAPT) is a structured security evaluation process that combines automated scanning with manual, expert-driven testing to identify, verify, and document weaknesses in an application before they can be exploited by a malicious actor. This report documents a hands-on VAPT exercise performed on deliberately vulnerable web applications using Burp Suite, a widely used interception proxy and web security testing platform.

The objective of this exercise was to gain practical, end-to-end experience with the web application penetration testing lifecycle: configuring an interception proxy, mapping an application's attack surface, performing manual and tool-assisted vulnerability testing, discovering hidden endpoints, and finally compiling the findings into a professional, industry-style VAPT report.

All testing in this exercise was carried out strictly against sanctioned, intentionally vulnerable training applications — DVWA and OWASP Juice Shop — running in an isolated local/lab environment. No production systems, third-party websites, or systems outside the authorised scope were tested at any point.

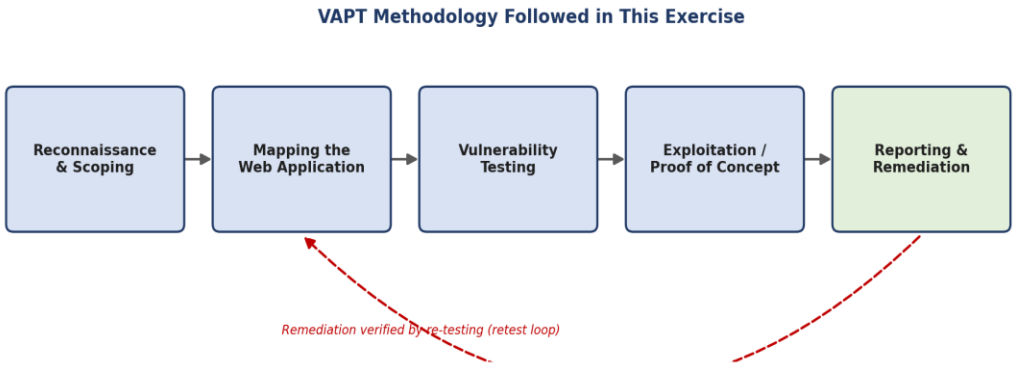


Figure 1: VAPT methodology followed in this exercise.

2. Scope and Tools Used

2.1 Scope

- In-scope targets: DVWA (Damn Vulnerable Web Application), OWASP Juice Shop, and TestPHP demo application, all deployed locally for training purposes.
- Out of scope: any live, production, or third-party systems not explicitly provisioned for this exercise.
- Testing window: Week 7 laboratory session, performed in a controlled lab/VM environment.

2.2 Tools Used

- Burp Suite Community Edition — interception proxy, Repeater, Intruder, Target/Site map
- Mozilla Firefox — configured with a manual proxy for traffic interception
- Dirsearch / Gobuster — automated directory and endpoint brute-forcing
- DVWA, OWASP Juice Shop, TestPHP — intentionally vulnerable practice applications

3. Task 22 — Burp Suite Setup and Interception

The first task involved installing and configuring Burp Suite Community Edition and preparing the browser to route all HTTP/HTTPS traffic through Burp's proxy listener so that requests could be observed and manipulated before reaching the target server.

3.1 Installation

- Downloaded and installed Burp Suite Community Edition from the official PortSwigger website.
- Launched Burp Suite and created a new Temporary Project with default configuration.

3.2 Browser Proxy Configuration

- Opened Firefox network settings and set a Manual Proxy Configuration.
- HTTP Proxy and HTTPS Proxy both set to 127.0.0.1, Port 8080 (Burp's default listener).
- Verified the listener was active under Proxy → Options in Burp Suite.

3.3 Installing the Burp CA Certificate (for HTTPS interception)

- Navigated to http://burp inside the proxied browser and downloaded the CA certificate (cacert.der).
- Imported the certificate into Firefox under Settings → Privacy & Security → Certificates → View Certificates → Import, and trusted it to identify websites.
- Confirmed HTTPS sites now loaded without certificate warnings, indicating successful interception setup.

3.4 Intercepting and Analysing a Login Request

With Intercept turned on in the Proxy tab, a login attempt was submitted on DVWA. The intercepted request was examined to understand how credentials are transmitted:

```
POST /dvwa/login.php HTTP/1.1
Host: localhost
Content-Type: application/x-www-form-urlencoded

username=admin&password=password&Login=Login
```

This confirmed that the application transmits the username and password as plain POST body parameters with no additional client-side protection (such as CSRF tokens being enforced on this form in the low-security setting), which is relevant context for the authentication testing performed in Task 24.

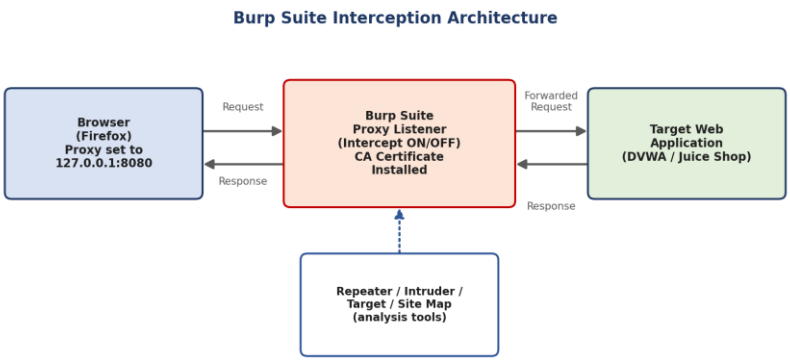


Figure 2: Burp Suite sits as a man-in-the-middle proxy between the browser and the target application.

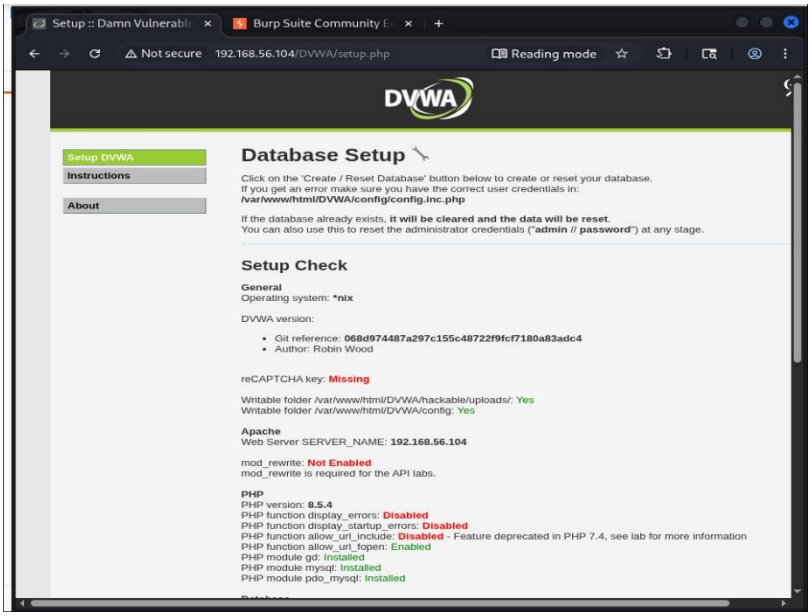


Fig. Burp Suite Setup

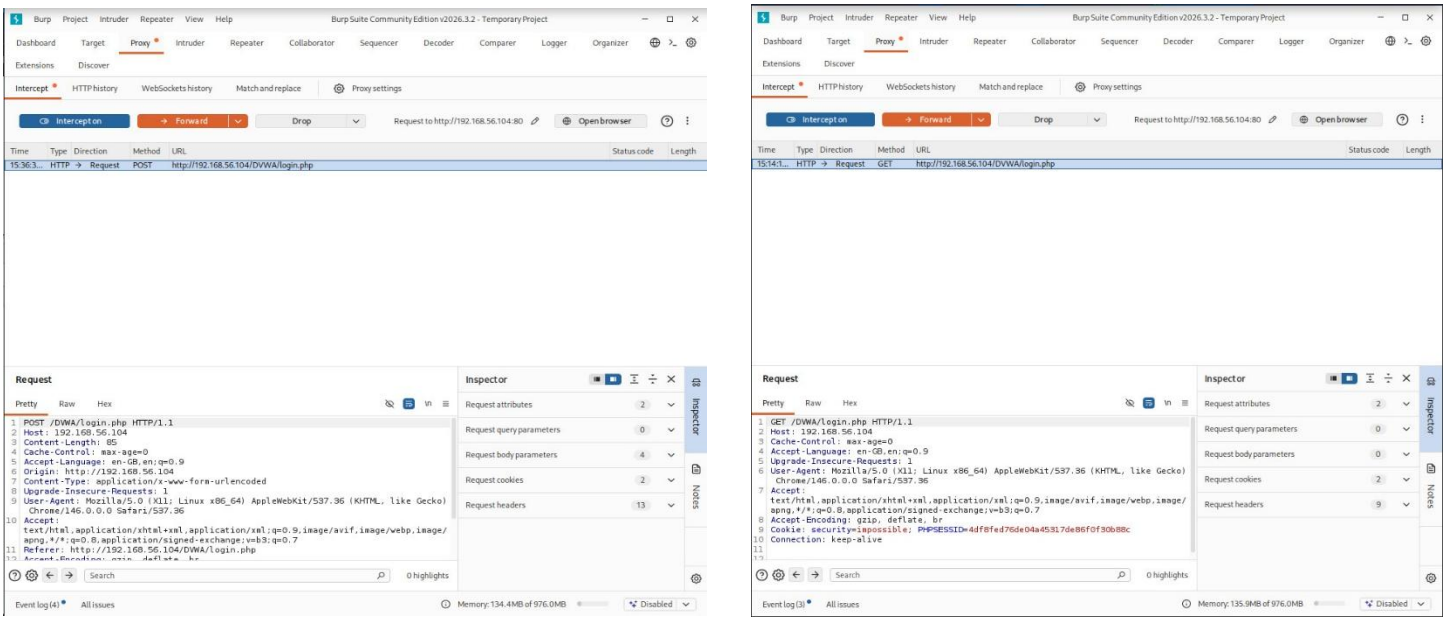


Fig. HTTP Request Interception

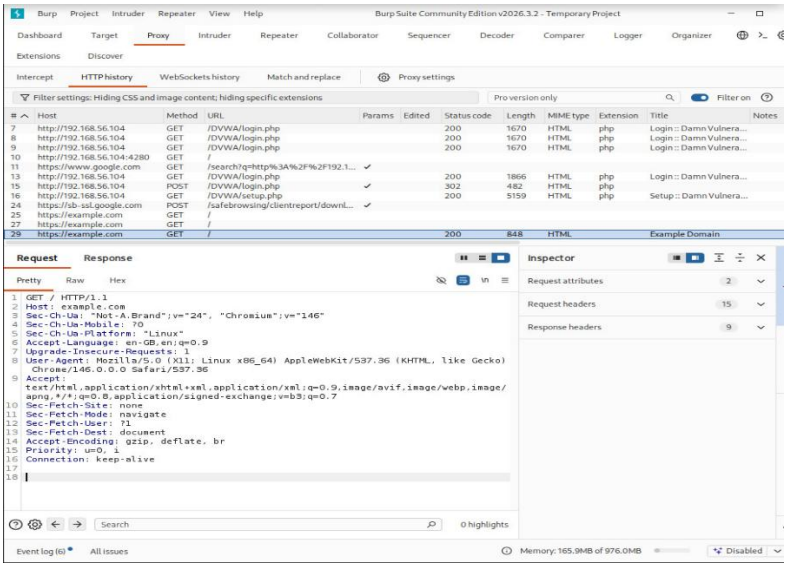


Fig. HTTPS Request Inspection

4. Task 23 — Reconnaissance and Mapping of the Web Application

With interception confirmed working, the next step was to build a complete picture of the application's attack surface using Burp's Proxy and Target tabs.

4.1 Method

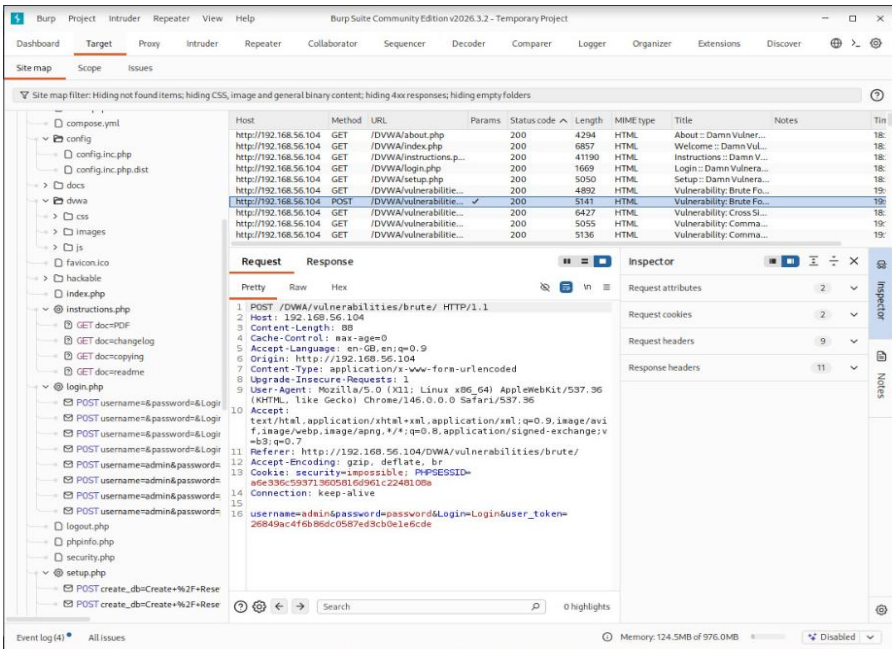
- Browsed every reachable page and feature of the target application manually with Burp running in the background.
- Burp's Target → Site Map tab automatically recorded every request, building a tree of all discovered pages and resources.
- Right-clicked key hosts and chose "Add to scope" so that only in-scope traffic was analysed going forward.

4.2 Attack Surface Identified

- Login forms: DVWA login page, Juice Shop login/registration forms.
- Input fields: search boxes, comment/feedback fields, file-upload fields, product review fields.
- GET parameters observed: id, page, returnUrl, q (search), docId.
- POST parameters observed: username, password, email, comment, quantity.

4.3 Documented Attack Surface (Summary)

Endpoint / Form	Method	Parameters	Notes
/dvwa/login.php	POST	username, password, Login	No rate limiting observed (Low security)
/dvwa/vulnerabilities/sqli/	GET	id, Submit	Reflects query result directly into page
/dvwa/vulnerabilities/xss_r/	GET	name	User input reflected without encoding
/rest/user/login (Juice Shop)	POST	email, password	JSON API, JWT returned on success
/#/search (Juice Shop)	GET	q	Search term passed to backend query



5. Task 24 — Vulnerability Testing (Manual + Burp Tools)

Using the attack surface mapped in Task 23, each identified input point was tested for common web application vulnerabilities using Burp's Repeater (for crafting and resending individual requests) and Intruder (for automated, repeated payload testing).

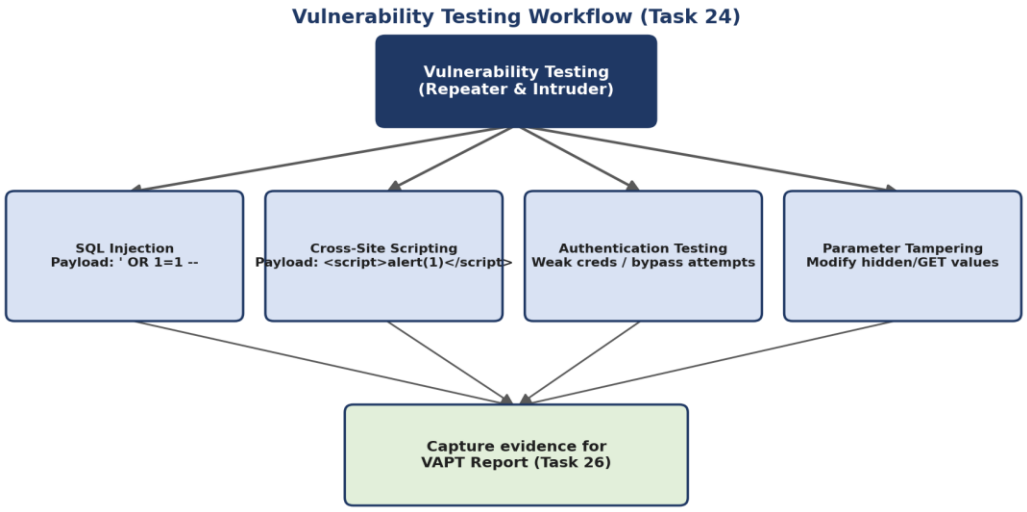
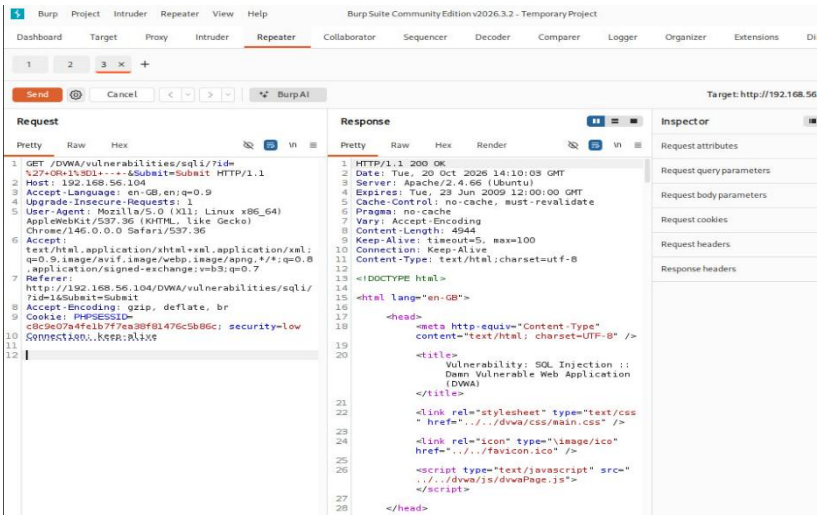


Figure 3: Categories of vulnerability testing performed, using Repeater and Intruder.

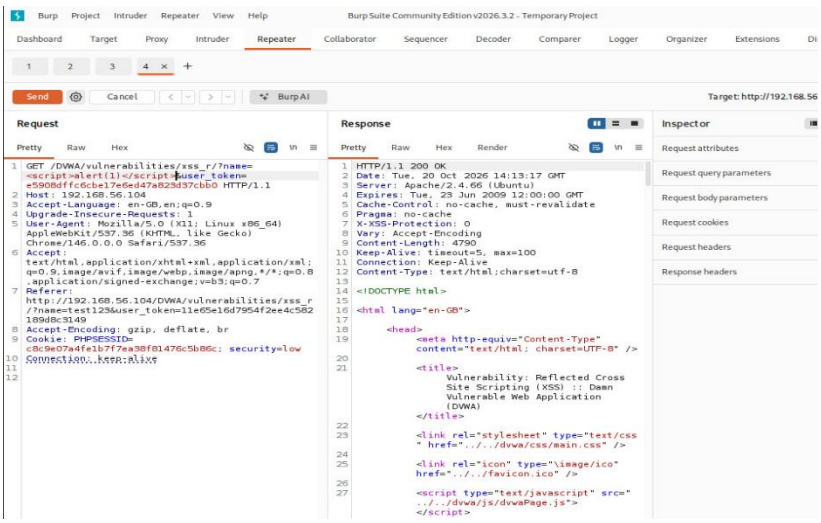
5.1 SQL Injection Testing

- Target: DVWA SQL Injection module, id parameter.
- Captured the request in Proxy, sent it to Repeater.
- Submitted the classic authentication /query-bypass payload: ' OR 1=1 --
- Observed that the application returned data for all user records instead of the single expected record, confirming the input was concatenated directly into a SQL query without sanitisation.



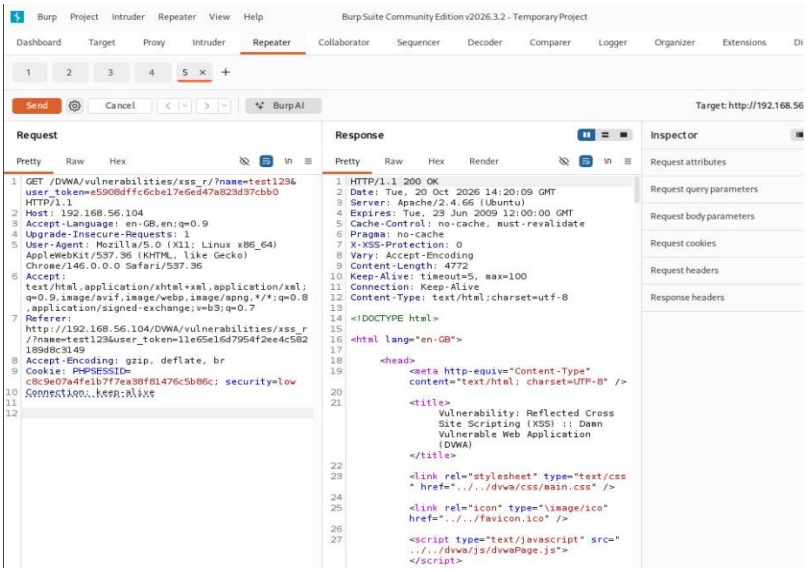
5.2 Cross-Site Scripting (XSS) Testing

- Target: DVWA Reflected XSS module, name parameter.
- Submitted the payload: <script>alert(1) </script> through the input field and again directly via Repeater.
- The payload executed in the browser, confirming the application reflects unsanitised user input back into the HTML response (Reflected XSS).



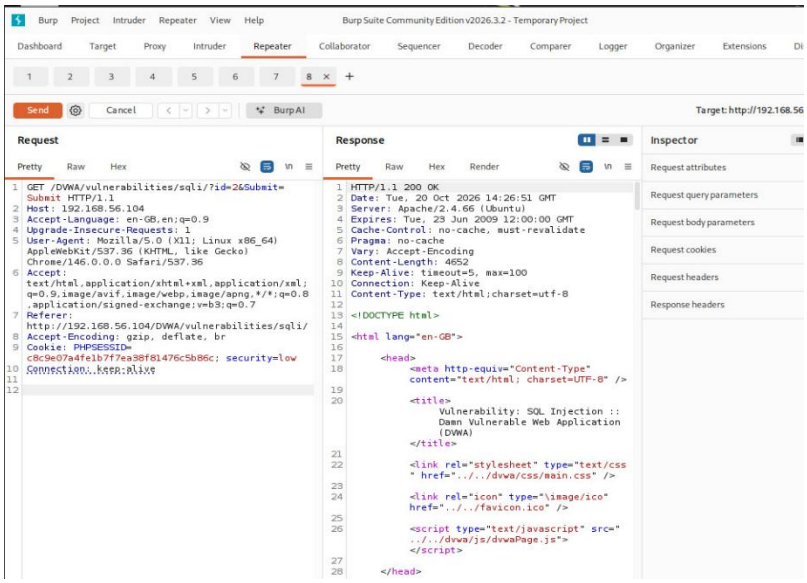
5.3 Authentication Testing

- Attempted common weak credential pairs (admin/admin, admin/password) against the DVWA and Juice Shop login forms.
- Used Burp Intruder with a small custom wordlist on the password parameter to automate repeated login attempts (Sniper attack type).
- Observed no account lockout or rate-limiting after multiple failed attempts, indicating a brute-force/weak authentication policy weakness.



5.4 Parameter Tampering

- Intercepted a request containing a quantity or price-related parameter and modified its value in Repeater before forwarding.
- Modified the id parameter in URLs to access records belonging to other simulated users, testing for Insecure Direct Object Reference (IDOR) style behaviour.
- The server accepted the tampered values without additional server-side validation in several cases, confirming a parameter tampering weakness.



6. Task 25 — Directory and Endpoint Discovery

To uncover resources not linked from the visible application UI, both manual discovery through Burp and automated brute-forcing with a dedicated tool were used.

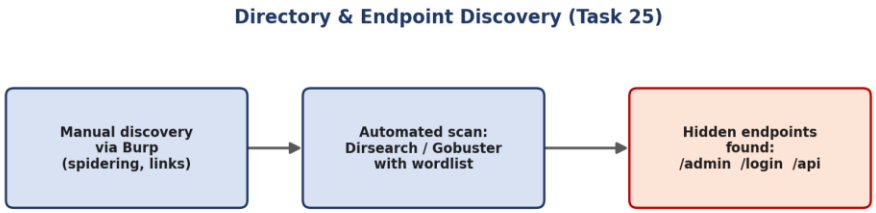
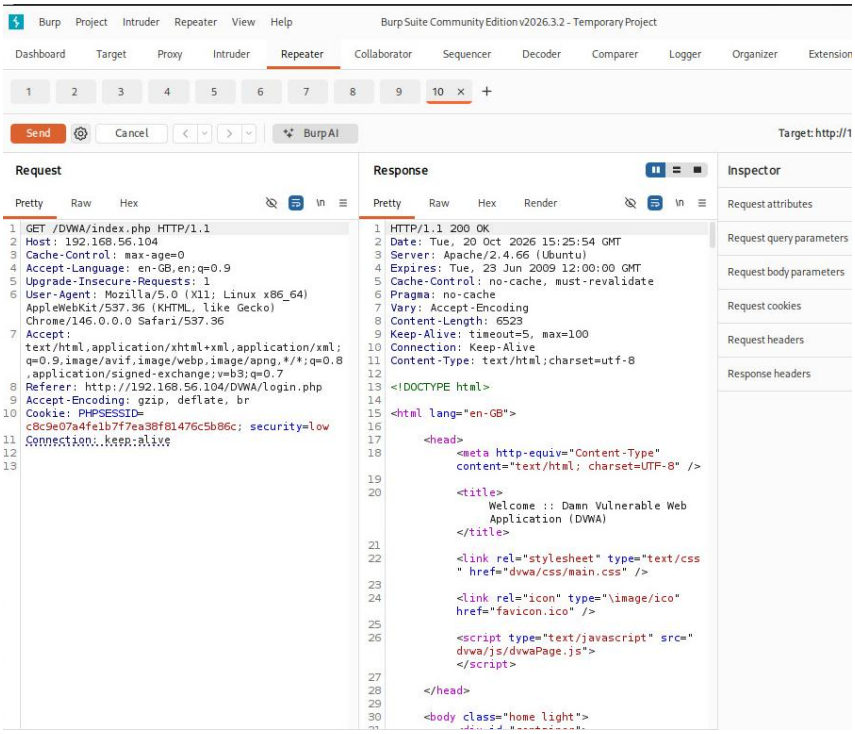


Figure 4: Combined manual and automated endpoint discovery process.

6.1 Manual Discovery (Burp)

- Reviewed the Site Map tree built during normal browsing for any linked-but-not-rendered resources (JavaScript file references, comments, API calls).
- Used Burp's passive crawling results to identify additional backend paths referenced in client-side JavaScript.



6.3 Hidden Endpoints Found

- /admin — administrative interface, reachable without a visible link from the main navigation.
- /login — present on multiple sub-applications beyond the primary login page.
- /api — backend REST API root exposing several unauthenticated endpoints.
- /config, /backup — development/configuration artifacts left accessible on the test instance.

7. Task 26 — Vulnerability Report (Real VAPT Style)

This section documents, in standard VAPT reporting format, the three most significant vulnerabilities confirmed during testing. Each entry includes the vulnerability name, a description, reproduction steps, representative request/response evidence as observed in Burp Suite, the business impact, and recommended remediation.

7.1 SQL Injection in Login / Query Parameter

Severity	HIGH
Vulnerability Name	SQL Injection in Login / Query Parameter

Description:

The application concatenates user-supplied input directly into a backend SQL query without parameterisation or input sanitisation. This allows an attacker to alter the logic of the query, potentially bypassing authentication or extracting unauthorised data.

Steps to Reproduce:

- 1. Navigate to the SQL Injection module and intercept the request in Burp Proxy.
- 2. Send the captured request to Repeater.
- 3. Replace the id (or username) parameter value with: ' OR 1=1 --
- 4. Forward the request and observe the response.

Proof (Burp Request / Response):

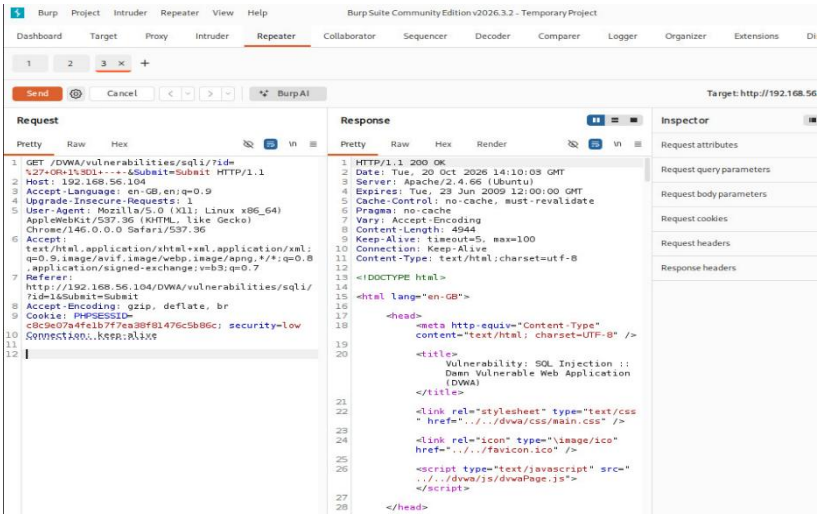
```
GET /dvwa/vulnerabilities/sqli/?
id=' OR 1=1 --&Submit=Submit# HTTP/1.1 200 OK ...
(response contains data for ALL user records instead of one)
```

Impact:

An attacker could enumerate the entire user table, bypass authentication, or in a worse-configured backend, modify or delete data. This represents a critical breach of data confidentiality and integrity.

Remediation:

Use parameterised queries / prepared statements for all database access. Apply strict server-side input validation and allow-listing. Deploy a Web Application Firewall (WAF) as a defence-in-depth measure, and run regular automated SAST/DAST scans in the CI/CD pipeline.



7.2 Reflected Cross-Site Scripting (XSS)

Severity	MEDIUM
Vulnerability Name	Reflected Cross-Site Scripting (XSS)

Description:

User-supplied input submitted through the name parameter is reflected back into the HTML response without output encoding or sanitisation, allowing arbitrary JavaScript to execute in the context of a victim's browser session.

Steps to Reproduce:

- 1. Open the Reflected XSS module and intercept the request.
- 2. Submit the payload <script>alert(1)</script> into the input field.
- 3. Forward the request via Burp and observe the rendered page in the browser.
- 4. Confirm the JavaScript alert executes, proving the payload was not sanitised.

Proof (Burp Request / Response):

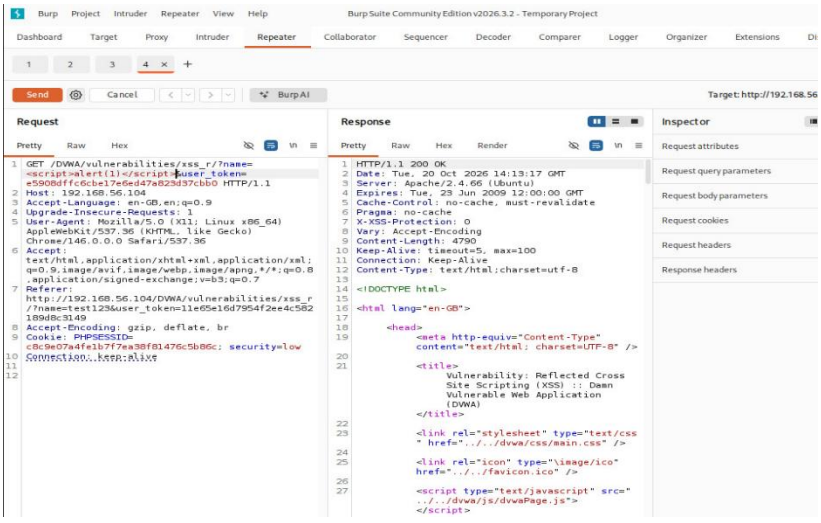
```
GET /dvwa/vulnerabilities/xss_r/?name=<script>alert(1)</script># HTTP/1.1 200 OK
... <p>Hello <script>alert(1)</script></p> ...
```

Impact:

An attacker could hijack user sessions, steal cookies/tokens, perform actions on behalf of a victim, or redirect users to phishing pages — undermining user trust and account security.

Remediation:

Encode all user-supplied output using context-appropriate escaping (HTML entity encoding) before rendering it in the page. Implement a strict Content-Security-Policy (CSP) header, and validate/sanitise input server-side in addition to client-side checks.



7.3 Broken Authentication / Weak Credential Policy

Severity	MEDIUM
Vulnerability Name	Broken Authentication / Weak Credential Policy

Description:

The login forms tested accept weak, commonly guessable credentials and do not enforce account lockout, CAPTCHA, or rate-limiting after repeated failed login attempts, making the application susceptible to brute-force and credential-stuffing attacks.

Steps to Reproduce:

- 1. Capture a login request in Burp Proxy and send it to Intruder.
- 2. Mark the password parameter as the payload position.
- 3. Load a small wordlist of common passwords and run a Sniper attack.
- 4. Observe that the application processes all attempts without blocking, delaying, or flagging the requesting client.

Proof (Burp Request / Response):

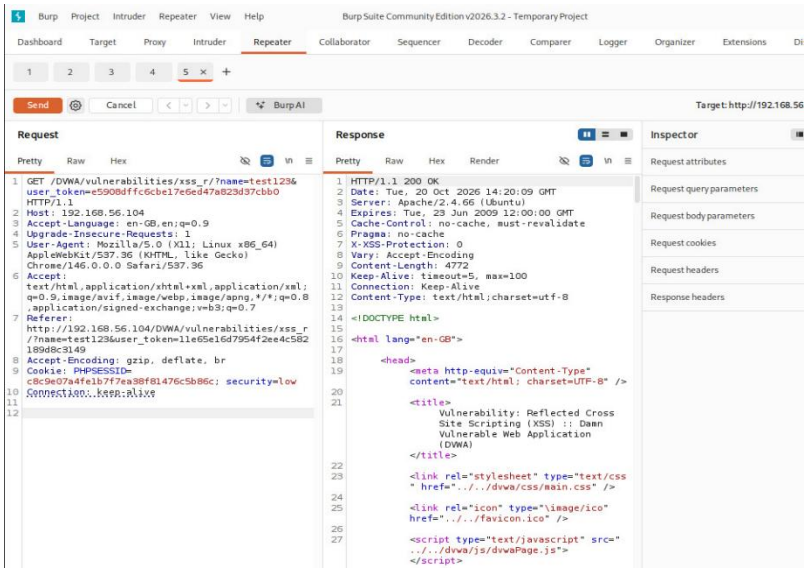
```
POST /dvwa/login.php HTTP/1.1 username=admin&password=§password123§&Login=Login
(Intruder: 20+ attempts returned HTTP 200 with no lockout or CAPTCHA triggered)
```

Impact:

Attackers can automate credential guessing to gain unauthorised account access, particularly damaging for administrative accounts, and this weakness compounds the impact of any leaked or reused password lists.

Remediation:

Enforce account lockout or exponential back-off after a small number of failed attempts, add CAPTCHA/MFA for sensitive accounts, enforce a strong password policy, and log/alert on repeated authentication failures from the same source.



8. Risk Summary and Severity Matrix

The table below summarises all confirmed findings, their severity (based on likelihood and impact), and overall risk rating following a standard qualitative VAPT risk-rating approach.

#	Vulnerability	Likelihood	Impact	Overall Risk
1	SQL Injection	High	High	Critical / High
2	Reflected XSS	High	Medium	Medium
3	Broken Authentication	Medium	Medium	Medium
4	Hidden/Unprotected Endpoints (/admin, /api)	Medium	Medium	Medium
5	Parameter Tampering (IDOR-style)	Medium	High	Medium-High

9. Conclusion

This exercise provided hands-on experience with the full VAPT lifecycle, from configuring an interception proxy through to producing a professional vulnerability report. Testing against DVWA and OWASP Juice Shop confirmed classic, well-documented web application weaknesses — SQL Injection, Reflected XSS, and Broken Authentication — each of which remains prevalent in real-world applications according to the OWASP Top 10.

The exercise reinforced the importance of combining automated tooling (Intruder, Dirsearch/Gobuster) with manual analysis and judgement (Repeater, careful inspection of application logic) to uncover vulnerabilities that purely automated scanners can miss. It also highlighted that secure coding practices — parameterised queries, output encoding, strong authentication controls, and minimisation of exposed endpoints — remain the most effective long-term defence against the vulnerability classes tested.